

HW 5: Dynamic Programming

Devin Fromond

Due: October 11th 2024

- Please type your solutions using $\text{\LaTeX}$ or any other software. Handwritten solutions will not be accepted.
- If we ask for a specific running time, a correct solution achieving it will receive full credit even if a faster solution exists. If no target running time is provided, faster solutions will earn more credit.

Problem 1: Road Trip (25 points)

You'd like to take a road trip from position 0 to position c_n , but your car can only go 300 miles at a time before needing to refuel. Fortunately, there are n checkpoints numbered 1 to n at specific positions on your path where you can refuel, $C = [c_1, c_2, \dots, c_n]$, which you can think of as points on a number line. As such, C will have **distinct positive values and will be sorted** and the last checkpoint will always be at c_n . However, each checkpoint at position c_i has an associated "penalty" p_i that it will cost you to use its gas pump (curiously, this is a one-time fee per checkpoint, and does not depend on how much gas you get) and these values will be given as an array $P = [p_1, p_2, \dots, p_n]$. c_n is always reachable by some series of checkpoints, and we always refuel in the checkpoint located at position c_n . We want to know the minimum total penalty to get to (and refuel at) to position c_n from position 0.

For example, if we have $C = [100, 250, 400]$, and $P = [12, 20, 3]$, then we can go from position 0 to checkpoint 1 at position c_1 and then to checkpoint 3 at position c_3 for a total penalty of $12 + 3 = 15$, which is the minimum penalty to get to c_3 . We could travel further initially by going from position 0 straight to c_2 (and then to c_3), but this isn't optimal as it costs us total penalty $20 + 3 = 23$.

Design a dynamic programming algorithm to efficiently find the minimum total penalty it will cost you to get to c_n from position 0 given C and P .

- (a) Define the entries of your table in words. E.g. $T[i]$ or $T[i][j]$ is ...

Solution: We will define $T[i]$ as the minimum total penalty to each checkpoint c_i from position 0, assuming we refuel at checkpoint c_i .

- (b) State base cases and recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct.

Solution: Base Case: We define $T[0]=0$ since it is the starting position. Therefore, we find $c_0 = 0$.

Recurrence Relation: We define $T[i]$ as the following:

$$T[i] = p_i + \min(T[j])$$

where $T[j]$ is subject to the constraints $0 \leq j < i$ and $c_i - c_j \leq 300$.

Correctness: We consider all previous checkpoints c_j where we can reach c_i while maintaining the distance is not greater than 300 miles. Additionally, among the valid c_j 's, we select the one with the minimum total penalty.

- (c) Write **pseudocode** for your algorithm to solve this problem.

Solution:

```
MinPenalty(C, N):
    initialize dp of size len(C) # T[0...len(C)]
    c[0] = 0 # base case
    T[0] = 0 # base case
    for i in range [1, len(C)]:
        T[i] = infinity
        for j in range [0, i - 1]:
            if c[i] - c[j] <= 300:
                T[i] = min(T[i], T[j] + P[i])
    return T[n]
```

(d) Analyze the running time of your algorithm.

Solution: We define $n = \text{len}(C)$. In our algorithm, we iterate over n twice. We additionally perform multiple $O(1)$ operations within the for loops and outside of the for loops. Therefore, We have $O(7(1) + n(n)) = O(n^2)$.

Problem 2: Difference Score (25 points)

You are given two strings X and Y of length n and m respectively. You want to find a difference score D_s which is defined to be the minimum number of steps to transform X to Y using the following options for each step:

1. Replace a character in the string with another character.
2. Add a character to the string.
3. Remove a character from the string.
4. Swap two adjacent characters.

Design an algorithm to compute D_s for two input strings.

Example: Given $X = WCA$ and $Y = WABC$, $D_s = 2$. ($WCA \rightarrow$ (swap) $WAC \rightarrow$ (insert B) $WABC$)

- (a) Define the entries of your table in words. E.g. $T[i]$ or $T[i, j]$ is ...

Solution: We will define $T[i, j]$ as the minimum number of steps required to transform the first i characters of the string X into the first j characters of the string Y .

- (b) State base cases and recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct.

Solution: Base Case: We define $T[0, 0] = 0$ since transforming an empty string to an empty string requires 0 steps. Similarly, we define $T[i, 0] = i$ and $T[0, j] = j$ since transforming the first i or j characters requires i deletions or j insertions, respectively.

Recurrence Relations: We have three recurrence relations - $X[i] = Y[j]$, $X[i] \neq Y[j]$, and swap.

$X[i] = Y[j]$: We will let $T[i, j] = T[i - 1, j - 1]$ since no operation is needed since characters match.

$X[i] \neq Y[j]$: We will define $T[i, j]$ as the following:

$$T[i, j] = \min \begin{cases} T[i - 1, j - 1] + 1 & \text{(Replace)} \\ T[i, j - 1] + 1 & \text{(Insert)} \\ T[i - 1, j] + 1 & \text{(Delete)} \end{cases}$$

Swap: We define the conditions for swap to be $X[i] = Y[j - 1]$ and $X[i - 1] = Y[j]$. If these two conditions are true, then we define $T[i, j]$ as the following:

$$T[i, j] = \min(T[i, j], T[i - 2, j - 2] + 1)$$

Correctness: We consider all possible operations and establish the conditions which

are necessary for them to occur. By taking the minimum of these possibilities, we ensure $T[i][j]$ is a minimum.

(c) Write **pseudocode** for your algorithm to solve this problem.

Solution:

```
diffScore(X, Y):
    initialize dp table of size (len(X))(len(Y)) # T[0...n][0...m]
    for i in range (0, len(X)):
        T[i][0]=i # base case
    for j in range (0, len(Y)):
        T[0][j]=j # base case
    for i in range (1, len(X)):
        for j in range (i, len(Y)):
            if X[i]==Y[j]:
                c = 0
            else:
                c = 1
            T[i][j]=min(
                T[i-1][j-1] + c, # replace/match
                T[i][j-1] + 1, # insert
                T[i-1][j] + 1 # delete
            )
            if X[i] == Y[j-1] AND X[i-1]==Y[j]:
                T[i][j] = min(T[i][j], T[i-2][j-2] + 1) # swap
    return T[n][m]
```

(d) Analyze the running time of your algorithm.

Solution: We will let $n = \text{len}(X)$ and $m = \text{len}(Y)$. We observe the base cases take $O(n + m)$ time. Additionally, we notice the nested loops iterate from 1 to i and 1 to j , thereby taking $O(mn)$. Therefore, we arrive at a final run time of $O(mn)$.

Problem 3: Maximum Pyramid Bananas (25 points)

A monkey is placed in given a pyramid of bananas stacks A , where $A[i][j]$ represents the number of bananas in the stack at position (i, j) . The monkey starts at position $(0, 0)$, and at each step, he can only go directly down-right or down-left to get to another stack. Additionally, for each down-right move that the monkey makes, he also obtains $bonus_r$ amount of bananas, and for each down-left move that the monkey makes, he also obtains $bonus_l$ amount of bananas. Design an algorithm to find the maximum amount of bananas the monkey can earn from the top of the pyramid to the bottom row. Consider the following example with $bonus_l = 1$ and $bonus_r = 2$:

3
7 4
2 4 6
8 5 9 3

(In array form, the input is given as $A = [[3], [7, 4], [2, 4, 6], [8, 5, 9, 3]]$)

The monkey's best path through this pyramid would be $(0, 0), (1, 0), (2, 1), (3, 2)$. It would collect $3 + 7 + 4 + 9 = 23$ bananas from the stacks and $1 + 2 + 2 = 5$ from the moves. The total amount of bananas obtained about therefore be $23 + 5 = \$28$.

- (a) Define the entries of your table in words. E.g. $T[i]$ or $T[i, j]$ is ...

Solution: We define $T[i][j]$ as the maximum total bananas the monkey can collect when reaching the position (i, j) .

- (b) State base cases and recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct.

Solution: Base Case: We observe $T[0][0] = A[0][0]$ since this is the monkey's starting position.

Recurrence Relations: We observe three primary recurrence relations: left, right, and middle positions.

Left Positions - $j = 0$: We define $T[i][0]$ as the following:

$$T[i][0] = T[i-1][0] + A[i][0] + bonus_l$$

since the monkey can only move down and to the left, given the constraint.

Right Positions - $j = i$: We define $T[i][i]$ as the following:

$$T[i][i] = T[i-1][i-1] + A[i][i] + bonus_r$$

since the monkey can only move down and to the right, given the constraint.

Middle Positions - $0 < j < i$: We define $T[i][j]$ as the following:

$$T[i][j] = \max(T[i-1][j-1] + A[i][j] + \text{bonus}_r, T[i-1][j] + A[i][j] + \text{bonus}_l)$$

since the monkey has two choices: move down and to the right or down and to the left. We choose the path which maximizes the number of bananas the monkey collects.

Correctness: With each move, the monkey maximizes the total bananas. By tracking and updating the total, we ensure any move is the ideal move with regard to the possible moves.

(c) Write **pseudocode** for your algorithm to solve this problem.

Solution:

```
maxPyramidBanana(A, bonus_l, bonus_r):
    # let n = rows in A
    initialize dp table of size (n)(n) with 0's # T[0...n][0...n]
    T[0][0]=A[0][0] # base case
    for i in range (1, n-1):
        for j in range (0, i):
            if j==0:
                T[i][0]=T[i-1][0]+A[i][0]+bonus_l # left pos
            else if j==i:
                T[i][i]=T[i-1][i-1]+A[i][i]+bonus_r # right pos
            else:
                max(
                    T[i-1][j]+A[i][j]+bonus_l, # left
                    T[i-1][j-1]+A[i][j] + bonus_r # right
                )
    max = T[n-1][0]
    for j in range (1, n):
        if T[n-1][j] > max:
            max = T[n-1][j]

    return max
```

(d) Analyze the running time of your algorithm.

Solution: We observe we can determine the number of elements in A given the number of rows.

Ultimately, we loop over the number of rows twice using a nested for loop. This results in a time complexity of $O(n^2)$. We loop once more over the rows to calculate the the greatest value in the final row - the maximum value. We could, in theory, simply track

this using a global variable, but I do not wish to alter my pseudocode. Nevertheless, even with this extra loop, we arrive at a final time complexity of $O(n^2)$.

Problem 4: Two Stonks (25 points)

Consider two possible investments A and B ; each year, an oracle tells you the return of the investments for the each of the n next years. This can be represented by arrays $A = [a_1, \dots, a_n]$ and $B = [b_1, \dots, b_n]$ (you can assume that these are always correct). At the beginning of each year i , you can place all your investment money entirely in A or entirely in B . It is guaranteed that your money should multiply by a_i (if you invested in A) or by b_i (if you invested in B) by the end of year i . However, if you move your money from A to B or B to A when transitioning to the next year, you lose half your money.

Given the return on investments for the next n years as arrays of multipliers $A = [a_1, \dots, a_n]$ and $B = [b_1, \dots, b_n]$, design an algorithm to determine the maximum multiplier you can achieve on your initial investment by the end of year n .

Example: Lets say that $A = [4, 2, 3, 1]$ and $B = [1, 2, 3, 3]$. You should place your money in investment A in years 1, 2, and 3 (to have your initial investment multiply by $4 \cdot 2 \cdot 3 = 24$). Then, you would transfer your money from A to B , which would cut your money in half (now you only have a 12x multiplier on your initial investment). By the end of year 4, you would achieve a 36x multiplier on your initial investment. Your algorithm should return 36 in this case.

Hint: a $1 \times n$ table will not be sufficient, but it doesn't have to be an $n \times n$ table!

- (a) Define the entries of your table in words. E.g. $T[i]$ or $T[i, j]$ is ...

Solution: We define $T[i][j]$ as the maximum multiplier possible on the initial investment by the end of the year i , given the investment amount is j at year i .

- (b) State base cases and recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct.

Solution: Base Case: We observe $T[0][0] = T[0][1] = 1$ since no investments are yet made.

Recurrence Relations: We observe two primary recurrence relations - investing in A and investing in B .

Investing in A - $j = 0$: We define $T[i][0]$ as the following:

$$T[i][0] = \max(a_i T[i-1][0], \frac{a_i T[i-1][1]}{2})$$

since we can either stay in A or switch from B to A and lose half the money.

Investing in B - $j = 1$: We define $T[i][1]$ as the following:

$$T[i][1] = \max(b_i T[i-1][1], \frac{b_i T[i-1][0]}{2})$$

since we can either stay in B or switch from A to B and lose half the money.

Correctness: We observe at each position, there are two means of arrival: staying in the current investment and switching investments. Therefore, by taking the maximum of these two options, we ensure the step chosen is ideal and yields the highest multiplier.

(c) Write **pseudocode** for your algorithm to solve this problem.

Solution:

```
twoStonks(A, B):
    # let n = len(A) = len(B)
    initialize dp table of size (n)(n) # T[0...n][0...n]
    T[0][0]=T[0][1]=1 # base case
    for i in range(1, n):
        stayA = a_i*T[i-1][0]
        switchA = 0.5*a_i*T[i-1][1]
        T[i][0]=max(stayA, switchA)

        stayB = b_i*T[i-1][1]
        switchB = 0.5*b_i*T[i-1][0]
        T[i][1] = max(stayB, switchB)

    return max(T[n][0], T[n][1])
```

(d) Analyze the running time of your algorithm.

Solution: We observe single for loop which iterates over n elements. Therefore, we have $O(n)$. Within the for loop, we perform multiple $O(1)$ operations. We arrive at a final time complexity of $O(n)$.